

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Monitoreo del estado de la batería con BroadcastReceiver en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	04	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	25/09/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Hanco Mullisaca Sergio Danilo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Ejercicio 1: Monitoreo de batería con BroadcastReceiver en Jetpack Compose Enunciado Implementar una pantalla en Jetpack Compose que muestre el nivel de batería en tiempo real mediante un BroadcastReceiver registrado dinámicamente con DisposableEffect. Desarrollo 1. Registro y Ciclo de Vida con DisposableEffect Se implementa el componente composable que escucha los cambios del sistema (ACTION_BATTERY_CHANGED) y calcula el porcentaje: <pre> 1 @Composable 2 fun BatteryScreen(modifier: Modifier = Modifier) { 3 var porcentaje by remember { mutableIntStateOf(0) } 4 val context = LocalContext.current 5 6 DisposableEffect(Unit) { 7 val receiver = object : BroadcastReceiver() { 8 override fun onReceive(context: Context?, intent: Intent?) { 9 val nivel = intent?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) ?: -1 10 val escala = intent?.getIntExtra(BatteryManager.EXTRA_SCALE, -1) ?: -1 11 if (nivel != -1 && escala != -1) { 12 porcentaje = (nivel * 100) / escala 13 } 14 } 15 } 16 context.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED)) 17 18 onDispose { 19 context.unregisterReceiver(receiver) 20 } 21 } </pre>

```
22
23     Text(text = "Batería: $porcentaje%", modifier = modifier)
24 }
```

2. Aspectos Clave

- **Extracción de datos:** Se obtienen EXTRA_LEVEL y EXTRA_SCALE para calcular el porcentaje de carga.
- **Gestión de recursos:** DisposableEffect registra el receiver al entrar en composición y lo desregistra en onDispose para evitar fugas de memoria.
- **Estado optimizado:** mutableIntStateOf(0) previene el autoboxing de tipos primitivos en recomposiciones frecuentes.

3. Evidencias de Ejecución y Pruebas

Se validó el comportamiento reactivo en tiempo real modificando el nivel de batería desde el emulador:

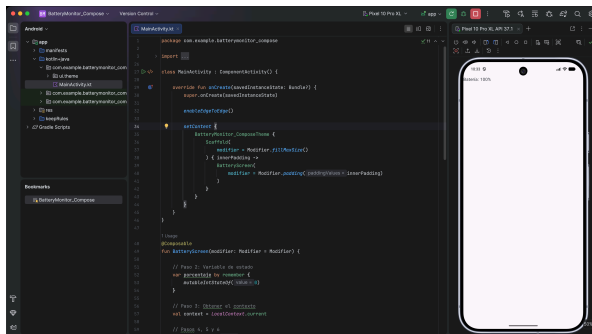


Figura 1: Ejecución inicial con nivel al 100%.

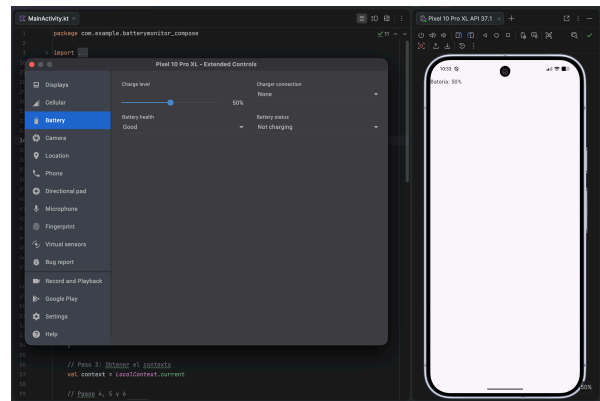


Figura 2: Simulación de nivel al 50%.

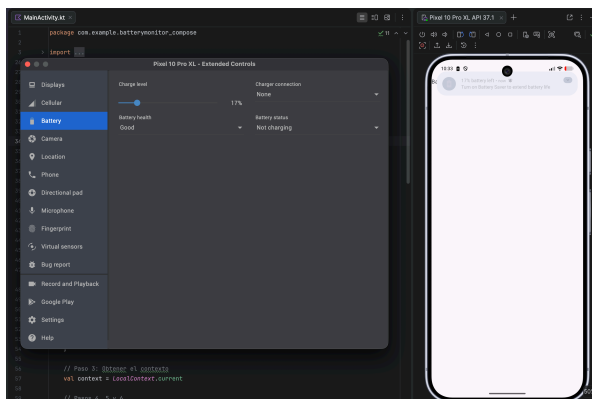


Figura 3: Simulación de nivel crítico al 17%.

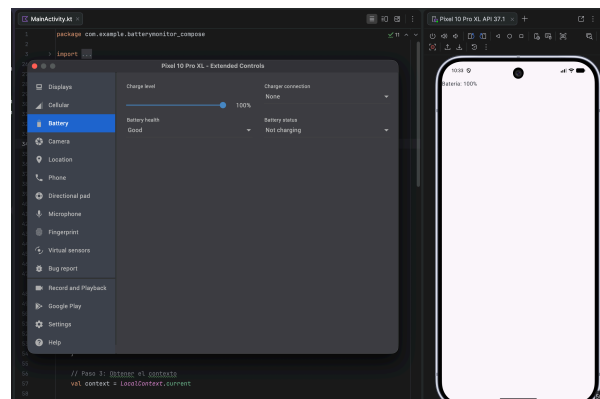


Figura 4: Restablecimiento de batería al 100%.

Ejercicio 2: Actualización manual del porcentaje de batería con broadcast personalizado

Enunciado

Tomando como referencia el ejercicio resuelto por el docente, agregar una segunda forma de actualizar el porcentaje de batería, disparada manualmente en lugar de por el evento automático del sistema:

1. Agregar un botón «Actualizar manualmente» a BatteryScreen que cree un Intent con una acción propia (com.tuapp.ACTUALIZAR_BATERIA), lo envuelva en un PendingIntent.getBroadcast(...) y lo active con PendingIntent.send().

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

2. Implementar un segundo BroadcastReceiver que escuche esa acción personalizada y, al recibirla, vuelva a leer el estado de la batería y actualice el mismo estado en pantalla.
3. Registrar y desregistrar este segundo receiver siguiendo el mismo patrón de DisposableEffect usado en el ejercicio resuelto.
4. **(Reto opcional)** Mostrar en pantalla, además del porcentaje, si la batería se está cargando o no, usando EXTRA_STATUS del Intent original.

Desarrollo y Solución Paso a Paso

Paso 1: Botón «Actualizar manualmente» y emisión con PendingIntent

Se define la acción propia y la función para encapsular el intent en un PendingIntent inmutable que dispara el broadcast bajo demanda al presionar el botón:

```

1 const val ACCION_ACTUALIZAR_BATERIA = "com.tuapp.ACTUALIZAR_BATERIA"
2
3 fun enviarActualizacionManual(context: Context) {
4     val intent = Intent(ACCION_ACTUALIZAR_BATERIA).apply {
5         setPackage(context.packageName) // Seguridad interna
6     }
7     val pendingIntent = PendingIntent.getBroadcast(
8         context,
9         0,
10        intent,
11        PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
12    )
13    pendingIntent.send()
14 }
```

Análisis: PendingIntent delega la ejecución al sistema; la bandera FLAG_IMMUTABLE impide que aplicaciones externas alteren el contenido del intent, mientras que setPackage() asegura que la transmisión ocurra exclusivamente dentro del paquete de la app.

Paso 2: Implementación del segundo BroadcastReceiver personalizado

Se implementa el segundo receptor encargado de capturar la acción manual y consultar síncronamente el estado del hardware:

```

1 val receiverManual = object : BroadcastReceiver() {
2     override fun onReceive(context: Context?, intent: Intent?) {
3         // Consulta instantánea del último broadcast sticky del sistema
4         val stickyIntent = context?.registerReceiver(null,
5             IntentFilter(Intent.ACTION_BATTERY_CHANGED))
6         val nivel = stickyIntent?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) ?: -1
7         val escala = stickyIntent?.getIntExtra(BatteryManager.EXTRA_SCALE, -1) ?: -1
8         if (nivel != -1 && escala != -1) {
9             porcentaje = (nivel * 100) / escala
10        }
11    }
12 }
```

Análisis: Al pasar null como receptor en registerReceiver(), Android retorna de inmediato el último intent pegajoso (sticky broadcast) sin crear una suscripción activa, permitiendo una lectura puntual y eficiente bajo demanda.

Paso 3: Registro y desregistro del segundo receptor con DisposableEffect

Se integra el segundo receiver en la jerarquía de Compose respetando su ciclo de vida:

```

1 DisposableEffect(Unit) {
2     val filtroManual = IntentFilter(ACCION_ACTUALIZAR_BATERIA)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

3     ContextCompat.registerReceiver(
4         context,
5         receiverManual,
6         filtroManual,
7         ContextCompat.RECEIVER_NOT_EXPORTED
8     )
9
10    onDispose {
11        context.unregisterReceiver(receiverManual)
12    }
13 }

```

Análisis: Acoplar la suscripción a DisposableEffect garantiza que el receptor se desregistre automáticamente al desmontarse la vista (onDispose), evitando fugas de memoria (**memory leaks**) en la aplicación.

Paso 4: (Reto opcional) Detección y visualización del estado de carga (EXTRA_STATUS)

Se extrae EXTRA_STATUS para evaluar si el dispositivo recibe energía eléctrica y reflejarlo en un indicador visual:

```

1 val status = intent?.getIntExtra(BatteryManager.EXTRA_STATUS, -1) ?: -1
2 val estaCargando = status == BatteryManager.BATTERY_STATUS_CHARGING ||
3                   status == BatteryManager.BATTERY_STATUS_FULL

```

Análisis: Es indispensable validar tanto BATTERY_STATUS_CHARGING como BATTERY_STATUS_FULL, ya que un dispositivo al 100% continúa enchufado a la corriente aunque el proceso de carga activa haya finalizado.

5. Evidencias de Ejecución y Pruebas

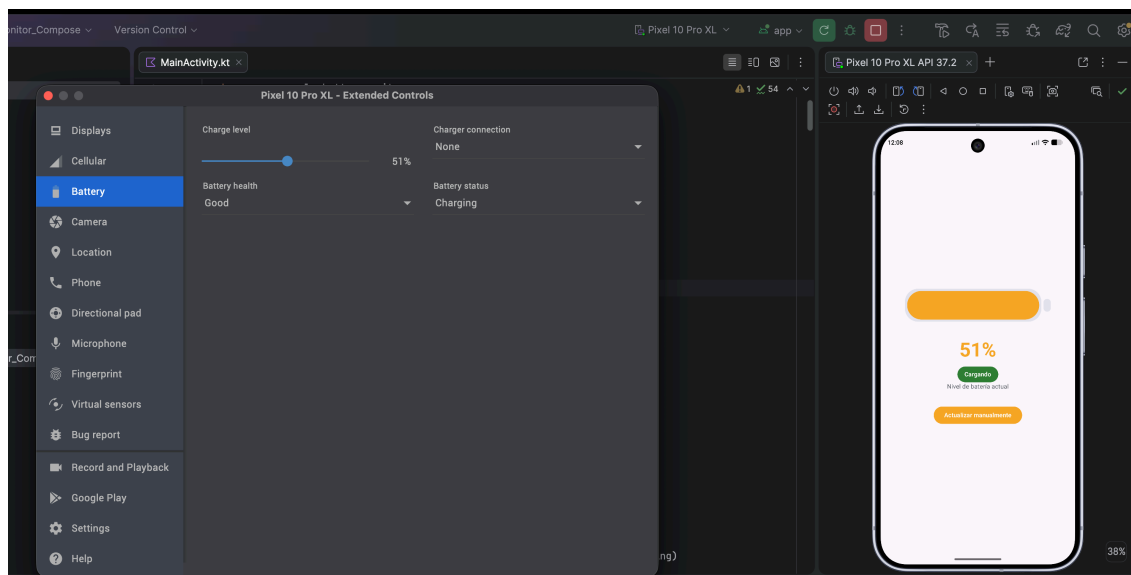


Figura 5: Simulación en Extended Controls al 51% en estado «Charging» reflejado con píldora verde «Cargando».

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

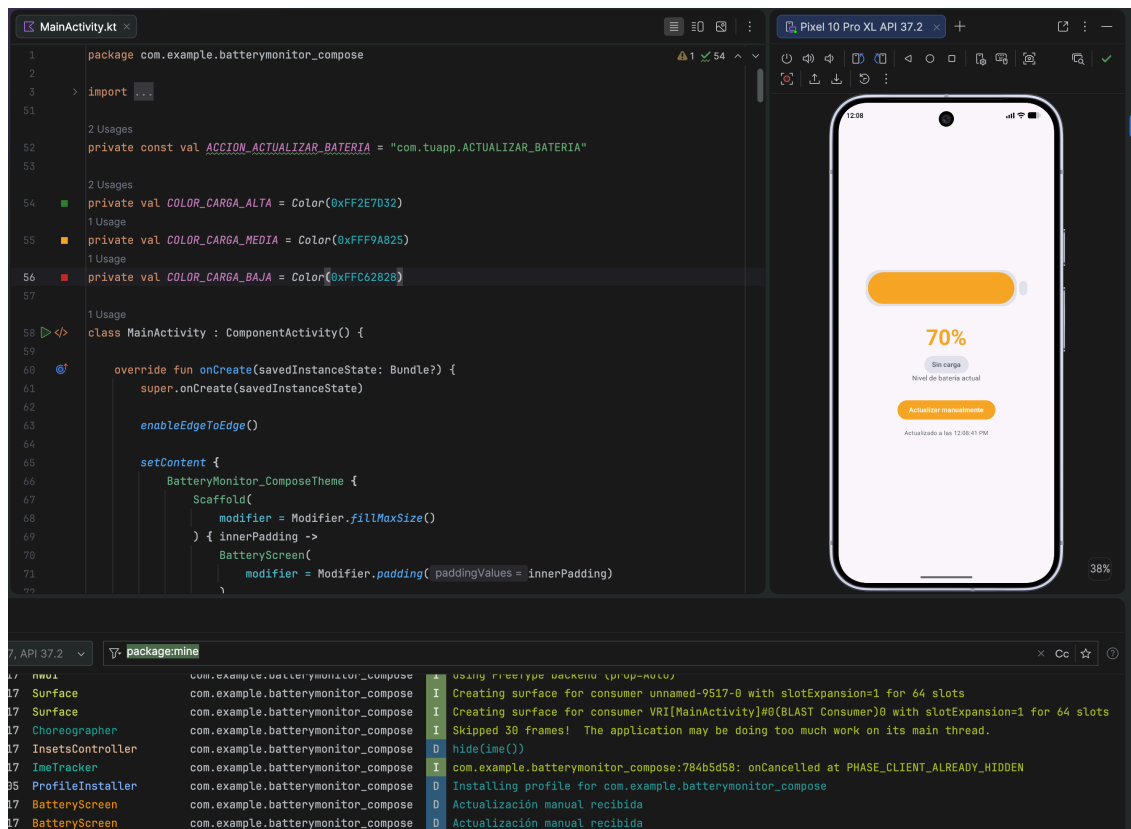


Figura 6: Actualización manual al 70% («Sin carga») y verificación del broadcast en Logcat («Actualización manual recibida»).

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1: ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantalla que muestra el porcentaje de batería en tiempo real)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Respuesta

Sí, logré resolver el ejercicio en su totalidad implementando el composible reactivo en Jetpack Compose. El sistema actualiza inmediatamente el porcentaje de carga ante emisiones del hardware sin reiniciar la interfaz.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

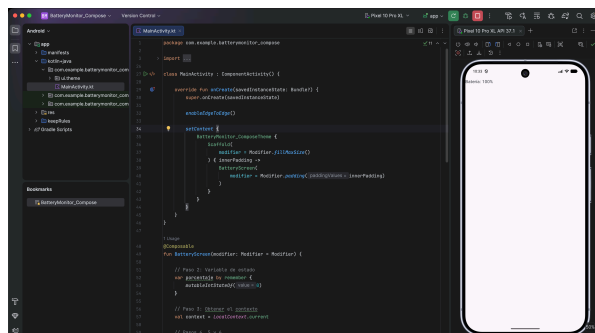


Figura 7: Ejecución inicial en Android Studio.

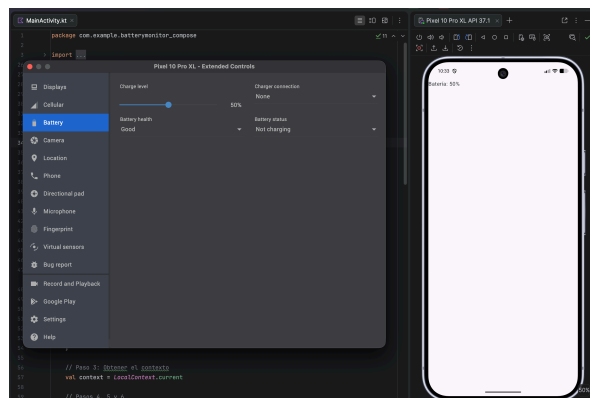


Figura 8: Simulación de respuesta al 50%.

(Nota: El registro completo de simulaciones y variaciones de nivel se encuentra detallado en la sección IV. Evidencias de Ejecución).

Pregunta 2: En tu propia implementación, ¿qué diferencia encontraste entre recibir el evento automático del sistema (ACTION_BATTERY_CHANGED) y disparar un broadcast manualmente con PendingIntent? ¿En qué situaciones usarías cada uno?

Respuesta

La principal diferencia radica en el control del flujo: el sistema emite ACTION_BATTERY_CHANGED de manera pasiva y automática, mientras que con PendingIntent decido el momento exacto en que se dispara la actualización bajo demanda.

En la práctica, el evento del sistema es sticky y entrega el estado actual de inmediato al suscribirse; en cambio, el broadcast manual requiere disparar explícitamente el intent para gatillar la lectura.

Utilizaría ACTION_BATTERY_CHANGED para paneles de monitoreo continuo o ahorro de energía, y el broadcast manual con PendingIntent para botones interactivos o refrescos solicitados directamente por el usuario.

Pregunta 3: En forma individual, comenta en el video de referencia Broadcast Receivers en Android con Jetpack Compose (YouTube) qué otras acciones del sistema, además del cambio de batería, se pueden escuchar con un BroadcastReceiver. (Colocar la captura del comentario; uno por integrante del grupo).

Respuesta

Realicé mi comentario destacando acciones esenciales del sistema como la transición al modo avión (ACTION_AIRPLANE_MODE_CHANGED), la conexión/desconexión a la corriente (ACTION_POWER_CONNECTED) y el cambio de conectividad de red.

Estas emisiones protegidas del sistema permiten que la aplicación reaccione inteligentemente al entorno del dispositivo sin necesidad de realizar sondeos constantes (**polling**), optimizando el uso de recursos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

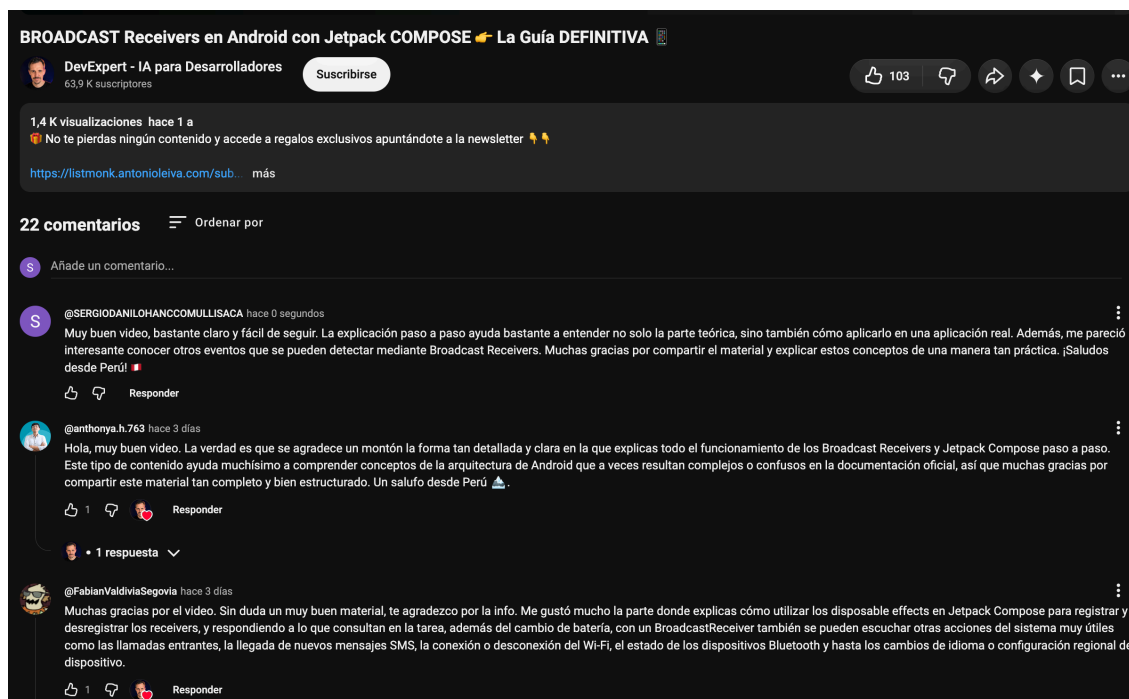


Figura 9: Captura del comentario realizado en el video de referencia de YouTube.

III. CONCLUSIONES

- El uso de DisposableEffect en Jetpack Compose vincula de manera segura el ciclo de vida de los BroadcastReceiver a la composición, garantizando el registro y desregistro automático en onDispose para evitar fugas de memoria.
- La combinación de receptores para eventos del sistema (ACTION_BATTERY_CHANGED) y acciones personalizadas enviadas mediante PendingIntent permite alternar entre monitoreo reactivo automático y consultas puntuales bajo demanda.
- La lectura de extras como EXTRA_LEVEL, EXTRA_SCALE y EXTRA_STATUS a través de broadcasts pegajosos (sticky broadcasts) permite obtener el estado completo del hardware (nivel y estado de carga) de forma eficiente.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.
- [2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

[3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.